

📖 Complete Code Explanation (Line-by-Line)

Let me break down **every single part** of your webhook code:

♦ 1. Imports Section

```
```python
from jwt import get_current_user
from fastapi import APIRouter, Request, HTTPException, Depends
from fastapi.responses import JSONResponse
from sqlalchemy.orm import Session
from database import get_db
from models.user import User
from config.config import STRIPE_WEBHOOK_SECRET
import stripe
import logging
```
```

| Import | Why It's Needed |
|-------------------------|--|
| `get_current_user` | Get authenticated user (not used here but imported) |
| `APIRouter` | Create a router for webhook endpoints |
| `Request` | Access raw HTTP request (needed for webhook signature) |
| `HTTPException` | Return error responses |
| `JSONResponse` | Return JSON back to Stripe |
| `Session` | Database session type hint |
| `get_db` | Get database connection |
| `User` | Access User model to update payment status |
| `STRIPE_WEBHOOK_SECRET` | Secret key to verify webhook is from Stripe |
| `stripe` | Stripe SDK to verify and process events |
| `logging` | Record events for debugging/monitoring |

♦ 2. Logging Setup

```
```python
Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
```
```

| Line | What It Does |
|------|--|
| 1 | Configure logging to show INFO level and above (INFO, WARNING, ERROR) |
| 2 | Create a logger object for this file ('__name__' = current module name) |

```
``python
webhook_router = APIRouter()
``
```

| Line | What It Does |
|---|---|
| <code>`webhook_router = APIRouter()`</code> | Creates a router to group webhook-related endpoints |

```

python
@webhook_router.post("/webhook")
async def webhook(request: Request, db: Session = Depends(get_db)):

```

| Line | What It Does |
|---|---|
| <pre> ----- </pre> | |
| <pre> `@webhook_router.post("/webhook")`</pre> | Defines a POST endpoint at <code>/webhook`</code> |
| <pre> `async def webhook(...)`</pre> | Async function to handle the request |
| <pre> `request: Request`</pre> | Gives access to raw request body and headers |
| <pre> `db: Session = Depends(get_db)`</pre> | Injects database session automatically |

> ****Why POST?**** Stripe sends data TO your server (not requesting data).

> 📌 ****Why async?**** Webhooks can be slow; async prevents blocking other requests.

💡 5. Get Raw Payload & Signature

```
```python
Get raw payload and signature
payload = await request.body()
sig_header = request.headers.get("Stripe-Signature")
```
```

| Line | What It Does |

|-----|-----|

| `payload = await request.body()` | Gets raw JSON data Stripe sent (as bytes) |

| `sig\_header = request.headers.get("Stripe-Signature")` | Gets signature Stripe sends to prove it's really them |

> 📌 ****Why raw body?**** Stripe signature verification needs the EXACT raw bytes, not parsed JSON.

> 📌 ****Why signature?**** Prevents hackers from faking payment events.

💡 6. Check Webhook Secret Exists

```
```python
✅ Check if webhook secret is set
if not STRIPE_WEBHOOK_SECRET:
 logger.error("STRIPE_WEBHOOK_SECRET is not set in config!")
 return JsonResponse(
 content={"error": "Webhook secret not configured"},
 status_code=500
)
```
```

| Line | What It Does |

|-----|-----|

| `if not STRIPE\_WEBHOOK\_SECRET:` | Checks if secret key exists in config |

| `logger.error(...)` | Logs error to console/file |

| `return JsonResponse(...)` | Returns error to Stripe (status 500 = server error) |

> 📌 ****Why?**** Without the secret, you can't verify Stripe's signature → security risk!

♦ 7. Verify Webhook Signature

```
```python
Verify webhook signature
try:
 event = stripe.Webhook.construct_event(
 payload, sig_header, STRIPE_WEBHOOK_SECRET
)
except ValueError as e:
 logger.error(f"Invalid payload: {e}")
 return JsonResponse(content={"error": "Invalid payload"}, status_code=400)
except stripe.error.SignatureVerificationError as e:
 logger.error(f"Invalid signature: {e}")
 return JsonResponse(content={"error": "Invalid signature"}, status_code=400)
```
```

| Line | What It Does |
|------|---|
| 1 | Verifies signature + parses JSON into event object |
| 2 | Catches if payload is malformed |
| 3 | Catches if signature doesn't match (possible attack!) |
| 4 | Logs the error for debugging |
| 5 | Tells Stripe the request was bad |

> 📌 ****Why try/except?**** If verification fails, you **MUST** return an error — don't process the event!

♦ 8. Extract Subscription Data

```
```python
Get subscription object (used in multiple events)
subscription = event["data"]["object"]
subscription_id = subscription.get("id")
customer_id = subscription.get("customer")
```
```

| Line | What It Does |
|------|--|
| 1 | Gets the main object from Stripe event (subscription, invoice, etc.) |
| 2 | Gets subscription ID (e.g., 'sub_123abc') |

| `subscription.get("customer")` | Gets customer ID (e.g., `cus\_456def`) |

> 📌 ****Why?**** You need these IDs to find the user in your database.

💡 9. EVENT 1: Subscription Created

```
```python
EVENT 1: First Login - Customer Subscription Created
if event["type"] == "customer.subscription.created":
 logger.info(f"NEW CUSTOMER! Subscription created: {subscription_id}")

 # Find user by Stripe customer ID
 user = db.query(User).filter(User.stripe_customer_id == customer_id).first()
 if user:
 logger.info(f"User {user.email} is now a valid customer!")
...```
```

| Line | What It Does |

|-----|-----|

| `if event["type"] == "customer.subscription.created":` | Checks if this is a new subscription event |

| `logger.info(...)` | Logs new customer for tracking |

| `db.query(User).filter(...)` | Finds user in your database by Stripe customer ID |

| `if user:` | Only logs if user exists (might not if created externally) |

> 📌 **\*\*When triggered?\*\*** When user first subscribes (first login).

> 📌 **\*\*Action?\*\*** Just logging here — actual activation happens in `invoice.payment\_succeeded`.

---

### ## 💡 10. EVENT 2: Payment Succeeded

```
```python
# EVENT 2: Payment Succeeded (Monthly Renewal)
elif event["type"] == "invoice.payment_succeeded":
    logger.info(f"Payment succeeded for subscription: {subscription_id}")

    user = db.query(User).filter(User.stripe_subscription_id == subscription_id).first()
    if user:
        user.is_active = True
        db.commit()
...```
```

```
...     logger.info(f"User {user.email} access maintained")
...
```

Line	What It Does
1	elif event["type"] == "invoice.payment_succeeded": Checks if payment was successful
2	logger.info(...) Logs successful payment
3	db.query(User).filter(...) Finds user by subscription ID
4	user.is_active = True **Activates user access**
5	db.commit() Saves changes to database
6	logger.info(...) Confirms user access is active

- > 📌 **When triggered?** First payment + every monthly renewal.
- > 📌 **Action?** Keep user active (is_active = True).

11. EVENT 3: Payment Failed

```
python
# EVENT 3: Payment Failed
elif event["type"] == "invoice.payment_failed":
    logger.warning(f"Payment FAILED for subscription: {subscription_id}")

    user = db.query(User).filter(User.stripe_subscription_id == subscription_id).first()
    if user:
        user.is_active = False
        db.commit()
        logger.warning(f"User {user.email} access blocked - payment failed")
...
```

Line	What It Does
1	elif event["type"] == "invoice.payment_failed": Checks if payment failed
2	logger.warning(...) Logs warning (needs attention)
3	user.is_active = False **Blocks user access**
4	db.commit() Saves changes to database
5	logger.warning(...) Confirms user is blocked

- > 📌 **When triggered?** Card declined, expired, insufficient funds.
- > 📌 **Action?** Block access (is_active = False).

♦ 12. EVENT 4: Subscription Deleted/Canceled

```
```python
EVENT 4: Subscription Canceled/Deleted
elif event["type"] == "customer.subscription.deleted":
 logger.warning(f"Subscription deleted: {subscription_id}")

 user = db.query(User).filter(User.stripe_subscription_id == subscription_id).first()
 if user:
 user.is_active = False
 db.commit()
 logger.warning(f"User {user.email} access blocked - subscription ended")
...
```
```

| Line | What It Does |
|------|---|
| 1 | Checks if subscription was canceled/deleted |
| 2 | Logs cancellation |
| 3 | Blocks user access |
| 4 | Saves changes to database |
| 5 | Confirms user is blocked |

- > 📌 **When triggered?** User manually cancels subscription.
- > 📌 **Action?** Block access (`is_active = False`).

♦ 13. Always Return Success to Stripe

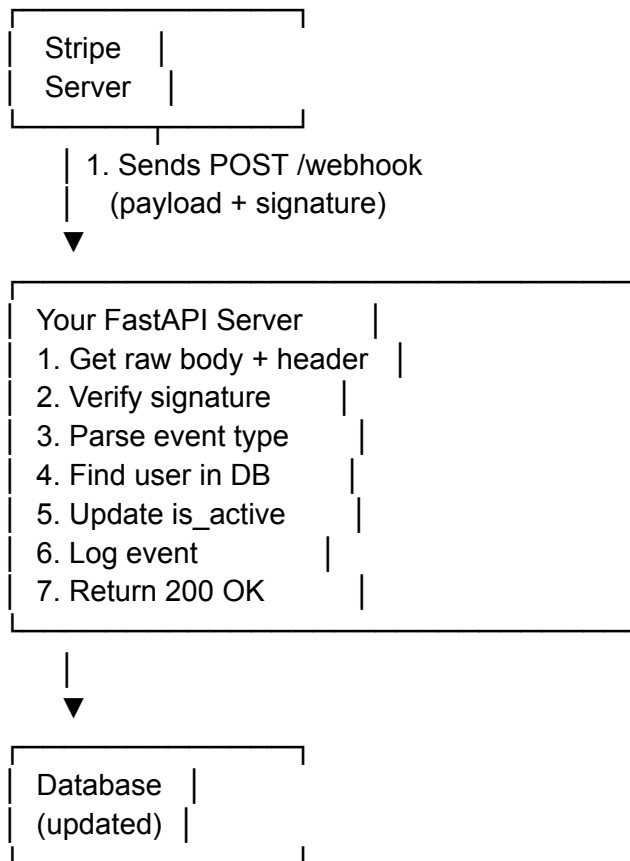
```
```python
Always return 200 to Stripe
return JsonResponse(content={"status": "success"})
...
```
```

| Line | What It Does |
|------|---|
| 1 | Sends response back to Stripe |
| 2 | Tells Stripe "I received this, don't retry" |

- > 📌 **Why?** If you don't return 200, Stripe will **retry the webhook multiple times** (can cause duplicate processing).
- > 📌 **Important:** Return 200 **even if your logic fails** — handle errors internally.

📊 Full Flow Diagram

...



...

✅ Summary Table

| Section | Purpose |
|----------------------------|------------------------------------|
| ----- | ----- |
| Imports | Get all tools needed |
| Logging Setup | Enable event tracking |
| Router | Group webhook routes |
| Endpoint | Define `/webhook` POST route |
| Payload + Signature | Get raw data from Stripe |
| Secret Check | Ensure config is correct |
| Signature Verify | Prove it's really Stripe |
| Event Handling | Process 4 different payment events |

| ****Return 200**** | Tell Stripe "got it, stop retrying" |

🎯 Key Takeaways

| Concept | Why It Matters |

|-----|-----|

| ****Raw body**** | Signature verification needs exact bytes |

| ****Signature**** | Prevents fake payment events |

| ****Logging**** | Debug + audit trail |

| ****Commit**** | Save user status to database |

| ****Return 200**** | Prevent Stripe from retrying endlessly |

Want me to explain any other file (user_handler, login flow, etc.)? 🚀

📖 Complete Code Explanation (Line-by-Line)

Let me break down ****every single part**** of your webhook code:

💡 1. Imports Section

```
```python
from jwt import get_current_user
from fastapi import APIRouter, Request, HTTPException, Depends
from fastapi.responses import JSONResponse
from sqlalchemy.orm import Session
from database import get_db
from models.user import User
from config.config import STRIPE_WEBHOOK_SECRET
import stripe
import logging
```
```

| Import | Why It's Needed |

|-----|-----|

| ``get_current_user`` | Get authenticated user (not used here but imported) |

| ``APIRouter`` | Create a router for webhook endpoints |

| ``Request`` | Access raw HTTP request (needed for webhook signature) |

| | |
|-------------------------|---|
| `HTTPException` | Return error responses |
| `JSONResponse` | Return JSON back to Stripe |
| `Session` | Database session type hint |
| `get_db` | Get database connection |
| `User` | Access User model to update payment status |
| `STRIPE_WEBHOOK_SECRET` | Secret key to verify webhook is from Stripe |
| `stripe` | Stripe SDK to verify and process events |
| `logging` | Record events for debugging/monitoring |

♦ 2. Logging Setup

```
```python
Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)
```
```

| Line | What It Does |
|---|---|
| ----- | |
| `logging.basicConfig(level=logging.INFO)` | Configure logging to show INFO level and above (INFO, WARNING, ERROR) |
| `logger = logging.getLogger(__name__)` | Create a logger object for this file (`__name__` = current module name) |

> 📌 ****Why?**** So you can use `logger.info()`, `logger.warning()`, `logger.error()` throughout the code.

♦ 3. Create Router

```
```python
webhook_router = APIRouter()
```
```

| Line | What It Does |
|--------------------------------|---|
| ----- | |
| `webhook_router = APIRouter()` | Creates a router to group webhook-related endpoints |

> 📌 ****Why?**** Keeps webhook routes separate from user/project routes. Later included in `main.py`.

♦ 4. Webhook Endpoint Definition

```
```python
@webhook_router.post("/webhook")
async def webhook(request: Request, db: Session = Depends(get_db)):
```
```

| Line | What It Does |
|------|--|
| 1 | Defines a POST endpoint at `/webhook` |
| 2 | Async function to handle the request |
| 3 | Gives access to raw request body and headers |
| 4 | Injects database session automatically |

- > 📌 **Why POST?** Stripe sends data TO your server (not requesting data).
- > 📌 **Why async?** Webhooks can be slow; async prevents blocking other requests.

♦ 5. Get Raw Payload & Signature

```
```python
Get raw payload and signature
payload = await request.body()
sig_header = request.headers.get("Stripe-Signature")
```
```

| Line | What It Does |
|------|---|
| 1 | Gets raw JSON data Stripe sent (as bytes) |
| 2 | Gets signature Stripe sends to prove it's really them |

- > 📌 **Why raw body?** Stripe signature verification needs the EXACT raw bytes, not parsed JSON.
- > 📌 **Why signature?** Prevents hackers from faking payment events.

♦ 6. Check Webhook Secret Exists

```
```python
```

```
 Check if webhook secret is set
if not STRIPE_WEBHOOK_SECRET:
 logger.error("STRIPE_WEBHOOK_SECRET is not set in config!")
 return JsonResponse(
 content={"error": "Webhook secret not configured"},
 status_code=500
)
...
```

Line	What It Does
1	if not STRIPE_WEBHOOK_SECRET:   Checks if secret key exists in config
2	logger.error(...)   Logs error to console/file
3	return JsonResponse(...)   Returns error to Stripe (status 500 = server error)

>  **\*\*Why?\*\*** Without the secret, you can't verify Stripe's signature → security risk!

---

## ## 7. Verify Webhook Signature

```
```python
# Verify webhook signature
try:
    event = stripe.Webhook.construct_event(
        payload, sig_header, STRIPE_WEBHOOK_SECRET
    )
except ValueError as e:
    logger.error(f"Invalid payload: {e}")
    return JsonResponse(content={"error": "Invalid payload"}, status_code=400)
except stripe.error.SignatureVerificationError as e:
    logger.error(f"Invalid signature: {e}")
    return JsonResponse(content={"error": "Invalid signature"}, status_code=400)
...
```
```

| Line | What It Does                                                                             |
|------|------------------------------------------------------------------------------------------|
| 1    | stripe.Webhook.construct_event(...)   Verifies signature + parses JSON into event object |
| 2    | ValueError   Catches if payload is malformed                                             |
| 3    | SignatureVerificationError   Catches if signature doesn't match (possible attack!)       |
| 4    | logger.error(...)   Logs the error for debugging                                         |
| 5    | status_code=400   Tells Stripe the request was bad                                       |

> 📌 **\*\*Why try/except?\*\*** If verification fails, you MUST return an error — don't process the event!

---

## ## 💡 8. Extract Subscription Data

```
```python
# Get subscription object (used in multiple events)
subscription = event["data"]["object"]
subscription_id = subscription.get("id")
customer_id = subscription.get("customer")
```
```

| Line | What It Does |

|-----|-----|

| `event["data"]["object"]` | Gets the main object from Stripe event (subscription, invoice, etc.) |

| `subscription.get("id")` | Gets subscription ID (e.g., `sub\_123abc`) |

| `subscription.get("customer")` | Gets customer ID (e.g., `cus\_456def`) |

> 📌 **\*\*Why?\*\*** You need these IDs to find the user in your database.

---

## ## 💡 9. EVENT 1: Subscription Created

```
```python
# EVENT 1: First Login - Customer Subscription Created
if event["type"] == "customer.subscription.created":
    logger.info(f"NEW CUSTOMER! Subscription created: {subscription_id}")

    # Find user by Stripe customer ID
    user = db.query(User).filter(User.stripe_customer_id == customer_id).first()
    if user:
        logger.info(f"User {user.email} is now a valid customer!")
```
```

| Line | What It Does |

|-----|-----|

| `if event["type"] == "customer.subscription.created":` | Checks if this is a new subscription event |

| `logger.info(...)` | Logs new customer for tracking |

| `db.query(User).filter(...)` | Finds user in your database by Stripe customer ID |

| `if user:` | Only logs if user exists (might not if created externally) |

> 📌 **\*\*When triggered?\*\*** When user first subscribes (first login).  
> 📌 **\*\*Action?\*\*** Just logging here — actual activation happens in  
`invoice.payment\_succeeded`.

---

## ## 💡 10. EVENT 2: Payment Succeeded

```
```python
# EVENT 2: Payment Succeeded (Monthly Renewal)
elif event["type"] == "invoice.payment_succeeded":
    logger.info(f"Payment succeeded for subscription: {subscription_id}")

    user = db.query(User).filter(User.stripe_subscription_id == subscription_id).first()
    if user:
        user.is_active = True
        db.commit()
        logger.info(f"User {user.email} access maintained")
...
```
```

| Line | What It Does                     |
|------|----------------------------------|
| 1    | Checks if payment was successful |
| 2    | Logs successful payment          |
| 3    | Finds user by subscription ID    |
| 4    | Activates user access            |
| 5    | Saves changes to database        |
| 6    | Confirms user access is active   |

> 📌 **\*\*When triggered?\*\*** First payment + every monthly renewal.  
> 📌 **\*\*Action?\*\*** Keep user active (`is\_active = True`).

---

## ## 💡 11. EVENT 3: Payment Failed

```
```python
# EVENT 3: Payment Failed
elif event["type"] == "invoice.payment_failed":
    logger.warning(f"Payment FAILED for subscription: {subscription_id}")

    user = db.query(User).filter(User.stripe_subscription_id == subscription_id).first()
    if user:
```

```

        user.is_active = False
        db.commit()
        logger.warning(f"User {user.email} access blocked - payment failed")
    ...

```

Line	What It Does
`elif event["type"] == "invoice.payment_failed":`	Checks if payment failed
`logger.warning(...)`	Logs warning (needs attention)
`user.is_active = False`	**Blocks user access**
`db.commit()`	Saves changes to database
`logger.warning(...)`	Confirms user is blocked

- > 📌 ****When triggered?**** Card declined, expired, insufficient funds.
- > 📌 ****Action?**** Block access (`is\_active = False`).

💡 12. EVENT 4: Subscription Deleted/Canceled

```

```python
EVENT 4: Subscription Canceled/Deleted
elif event["type"] == "customer.subscription.deleted":
 logger.warning(f"Subscription deleted: {subscription_id}")

 user = db.query(User).filter(User.stripe_subscription_id == subscription_id).first()
 if user:
 user.is_active = False
 db.commit()
 logger.warning(f"User {user.email} access blocked - subscription ended")
 ...

```

Line	What It Does
`elif event["type"] == "customer.subscription.deleted":`	Checks if subscription was canceled/deleted
`logger.warning(...)`	Logs cancellation
`user.is_active = False`	**Blocks user access**
`db.commit()`	Saves changes to database
`logger.warning(...)`	Confirms user is blocked

- > 📌 **\*\*When triggered?\*\*** User manually cancels subscription.
- > 📌 **\*\*Action?\*\*** Block access (`is\_active = False`).

---

### ## 💡 13. Always Return Success to Stripe

```
```python
# Always return 200 to Stripe
return JSONResponse(content={"status": "success"})
```
```

| Line | What It Does                                                            |
|------|-------------------------------------------------------------------------|
| 1    | return JSONResponse(...)   Sends response back to Stripe                |
| 2    | status_code=200 (default)   Tells Stripe "I received this, don't retry" |

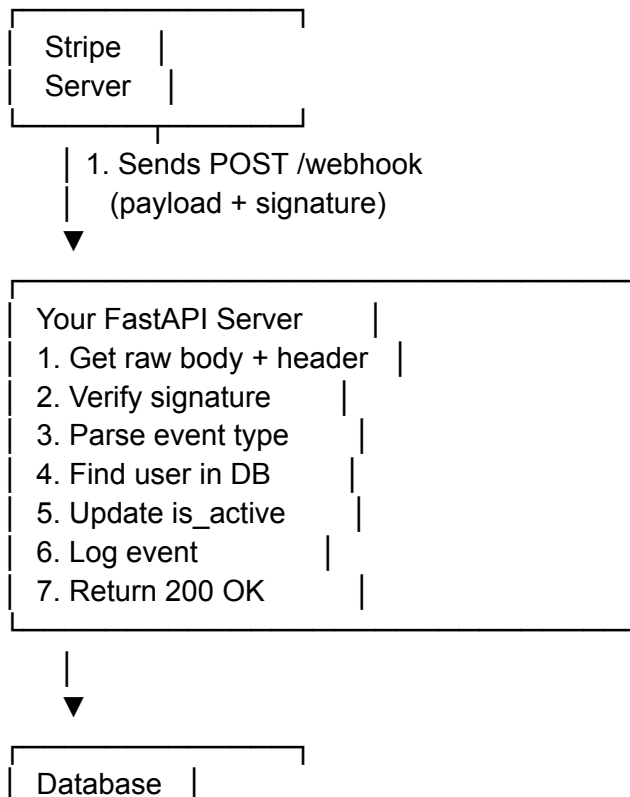
> 📌 **\*\*Why?\*\*** If you don't return 200, Stripe will **\*\*retry the webhook multiple times\*\*** (can cause duplicate processing).

> 📌 **\*\*Important:\*\*** Return 200 **\*\*even if your logic fails\*\*** — handle errors internally.

---

### ## 📊 Full Flow Diagram

...





| (updated) |

...

---

## ## ✅ Summary Table

| Section             | Purpose                             |
|---------------------|-------------------------------------|
| Imports             | Get all tools needed                |
| Logging Setup       | Enable event tracking               |
| Router              | Group webhook routes                |
| Endpoint            | Define `/webhook` POST route        |
| Payload + Signature | Get raw data from Stripe            |
| Secret Check        | Ensure config is correct            |
| Signature Verify    | Prove it's really Stripe            |
| Event Handling      | Process 4 different payment events  |
| Return 200          | Tell Stripe "got it, stop retrying" |

---

## ## 🎯 Key Takeaways

| Concept    | Why It Matters                           |
|------------|------------------------------------------|
| Raw body   | Signature verification needs exact bytes |
| Signature  | Prevents fake payment events             |
| Logging    | Debug + audit trail                      |
| Commit     | Save user status to database             |
| Return 200 | Prevent Stripe from retrying endlessly   |

---

Want me to explain any other file (user\_handler, login flow, etc.)? 🚀